```
import numpy as np
```

```
import numpy as np

# Union
def fuzzy_union(A, B):
    return np.maximum(A, B)

# Intersection
def fuzzy_intersection(A, B):
    return np.minimum(A, B)

# Complement
def fuzzy_complement(A):
    return 1 - A

# Difference
def fuzzy_difference(A, B):
    return np.minimum(A, 1 - B)

# Cartesian Product
def cartesian_product(A, B):
    return np.minimum.outer(A, B)

# Max-Min Composition
def max_min_composition(R, S):
    result = np.zeros((R.shape[0], S.shape[1]))

    for i in range(R.shape[0]):
        for j in range(S.shape[1]):
            result[i][j] = np.max(
                np.minimum(R[i, :], S[:, j])
            )

    return result
```

```
# Example fuzzy sets
A = np.array([0.2, 0.4, 0.6, 0.8])
B = np.array([0.3, 0.5, 0.7, 0.9])

print("Union:", fuzzy_union(A, B))
print("Intersection:", fuzzy_intersection(A, B))
print("Complement of A:", fuzzy_complement(A))
print("Difference:", fuzzy_difference(A, B))
```

```
Union: [0.3 0.5 0.7 0.9]
Intersection: [0.2 0.4 0.6 0.8]
Complement of A: [0.8 0.6 0.4 0.2]
Difference: [0.2 0.4 0.3 0.1]
```

```
# Fuzzy relations
R = np.array([[0.2, 0.5],
              [0.4, 0.7]])

S = np.array([[0.6, 0.3],
              [0.8, 0.5]])
```

```
print("\nCartesian Product:")
print(cartesian_product(A[:2], B[:2]))

print("\nMax-Min Composition:")
print(max_min_composition(R, S))
```

```
Cartesian Product:
[[0.2 0.2]
 [0.3 0.4]]

Max-Min Composition:
[[0.5 0.5]
 [0.7 0.5]]
```